DocRAG — Document Question-Answering System

From Proof-of-concept to Production-Grade RAG (Retrieval-Augmented Generation) backend built with Django REST Framework, PGVector, Celery, and Ollama. Upload documents, generate embeddings asynchronously, and ask natural-language questions against your own document library.

Table of Contents

Architecture

```
Client
  │
  ▼
Django REST Framework  ──►  JWT Auth (SimpleJWT + HTTP-only cookie)
  │
  ├── POST /api/documents/   ──►  Celery Worker  ──►  Ollama (embed)  ──►
PGVector
  │
  └── POST /api/qa/ask/      ──►  Celery Worker
                                    ├── Ollama (embed query)
                                    ├── PGVector (similarity search)
                                    ├── BM25 + LLM hybrid rerank
                                    └── Ollama (generate answer)
                                  GET /api/qa/result/{task_id}/  ──►  Celery
result backend
```

Document processing and question answering are both fully asynchronous. After upload the API returns immediately while a Celery worker chunks the file and writes embeddings into PGVector. The `/status/`

endpoint lets clients poll until processing is `completed`. Question answering follows the same pattern — `POST /api/qa/ask/` returns a `task_id` and the client polls `/api/qa/result/{task_id}/` for the answer.

Tech Stack

Layer	Technology
API framework	Django 5.0 + Django REST Framework
Auth	SimpleJWT (access token in body, refresh token in HTTP-only cookie, blacklisting)
Database	PostgreSQL 16 with `pgvector` extension
Vector store	LangChain `langchain-postgres` (PGVector)
Embeddings / LLM	Ollama (local models — configurable via env)
Task queue	Celery + Redis
Task result backend	`django-celery-results`
Scheduled tasks	`django-celery-beat`
Filtering / search	`django-filters`
API schema	`drf-spectacular` (Swagger UI + ReDoc)
Testing	pytest + pytest-django, 99 tests, 84% coverage

Project Structure

```
backend/
├── core/                        # Django project config
│   ├── settings/
│   │   ├── base.py              # Shared settings (all environments inherit
from this)
│   │   ├── env.py               # django-environ bootstrapping, single source
of truth
│   │   ├── local.py             # Development overrides
│   │   ├── production.py        # Production hardening
│   │   └── test.py              # Test runner settings
│   ├── exceptions.py            # AppError base + custom_exception_handler
│   ├── urls.py
│   ├── celery.py
│   └── wsgi.py
│
├── accounts/                    # Custom User model, JWT auth
│   ├── exceptions.py            # AccountsServiceError hierarchy
```

```
│   ├── selectors.py            # Read-only DB queries
│   ├── services/
│   │   └── user.py             # register_user, update_user_profile,
logout_user
│   ├── serializers.py          # Input/Output split (RegisterInputSerializer,
UserOutputSerializer, …)
│   ├── views.py                # Thin views — delegate to services, full
OpenAPI docs
│   ├── models.py
│   └── urls.py
│
├── documents/                  # Document upload, chunking, embedding
pipeline
│   ├── exceptions.py           # DocumentServiceError hierarchy
│   ├── selectors.py            # document_get, document_list,
document_exists_for_user
│   ├── services/
│   │   ├── document.py         # CreateDocumentService,
DeleteDocumentService, …
│   │   └── embedding.py       # Embedding pipeline logic
│   ├── serializers.py          # DocumentUploadInputSerializer,
DocumentOutputSerializer
│   ├── views.py
│   ├── tasks.py                # Thin Celery task — delegates to services
│   ├── filters.py
│   ├── models.py
│   └── urls.py
│
├── qa/                         # Question answering, streaming, activity log
│   ├── exceptions.py           # QAServiceError hierarchy
│   ├── selectors.py            # question_activity_list,
question_activity_stats
│   ├── services/
│   │   ├── ask.py              # AskQuestionService — validates and
dispatches to Celery
│   │   ├── process.py          # ProcessQuestionService — full RAG pipeline
│   │   ├── embedding.py        # Query embedding with Redis cache
│   │   ├── retrieval.py        # PGVector retrieval
│   │   ├── reranking.py        # BM25 + LLM hybrid reranking
│   │   ├── stream.py           # Streaming pipeline
│   │   └── activity.py         # QuestionActivity CRUD services
│   ├── serializers.py
│   ├── views.py
│   ├── tasks.py                # Thin Celery task — delegates to
ProcessQuestionService
│   ├── filters.py
│   ├── pagination.py
│   ├── models.py
│   └── urls.py
│
└── tests/
    ├── conftest.py
    ├── fixtures/
```

```
    ├── test_accounts.py
    ├── test_documents.py
    ├── test_qa.py
    ├── test_integration.py
    └── test_workflow.py
```

Getting Started

Prerequisites

- Docker & Docker Compose, **or** Python 3.12+, PostgreSQL 16, Redis
- Ollama running locally with the following models pulled:

```
ollama pull embeddinggemma
ollama pull gemma4
```

Environment Variables

Copy `.env.example` to `.env` and fill in the values:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0

CORS_ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173
CSRF_TRUSTED_ORIGINS=http://localhost:3000,http://localhost:5173
REFRESH_COOKIE_SAMESITE=Lax

# PostgreSQL — used by docker-compose to provision the DB and by DATABASE_URL
POSTGRES_DB=ragdb
POSTGRES_USER=raguser
POSTGRES_PASSWORD=ragpassword
DATABASE_URL=postgresql://raguser:ragpassword@db:5432/ragdb

# Redis / Celery
REDIS_URL=redis://redis:6379/0

# Ollama — host.docker.internal reaches your host machine from inside Docker
OLLAMA_BASE_URL=http://host.docker.internal:11434
OLLAMA_EMBED_MODEL=embeddinggemma
OLLAMA_CHAT_MODEL=gemma4
OLLAMA_TIMEOUT=30

# JWT lifetime
ACCESS_TOKEN_LIFETIME_MINUTES=60
REFRESH_TOKEN_LIFETIME_DAYS=7
```

```
# Retrieval / chunking
CHUNK_SIZE=500
CHUNK_OVERLAP=50
TOP_K_CHUNKS=5
TOP_K_CHUNKS_PER_DOC=3

# Reranking
RERANK_TOP_K=5
RERANK_BM25_WEIGHT=0.4
RERANK_LLM_WEIGHT=0.6
RERANK_TOP_K_PER_DOC=3
RERANK_TOP_K_MAX=10
```

Running with Docker Compose

```
docker-compose up --build
```

This starts PostgreSQL (with pgvector), Redis, the Django API (gunicorn), and three Celery services (worker, beat, and the Django process). Migrations and static file collection run automatically on container start.

```
# Create a superuser for the admin panel
docker-compose exec web python manage.py createsuperuser
```

Running Locally (without Docker)

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Set the settings module
export DJANGO_SETTINGS_MODULE=core.settings.local

# 3. Apply migrations
python manage.py migrate

# 4. Start the API
python manage.py runserver

# 5. Start Celery worker (separate terminal)
celery -A core worker -l info -Q celery,documents,qa

# 6. (Optional) Start Celery Beat for scheduled tasks
celery -A core beat -l info --scheduler
django_celery_beat.schedulers:DatabaseScheduler
```

API Reference

Interactive Swagger UI is available at `http://localhost:8000/api/docs/` and ReDoc at `http://localhost:8000/api/redoc/`.

All endpoints except `register` and `login` require:

```
Authorization: Bearer <access_token>
```

Auth

Method	Endpoint	Description
`POST`	`/api/auth/register/`	Create a new user account
`POST`	`/api/auth/login/`	Login — returns `access` token in body, `refresh` token in HTTP-only cookie
`POST`	`/api/auth/refresh/`	Rotate access token using the refresh cookie
`POST`	`/api/auth/logout/`	Blacklist the refresh token from the cookie
`GET`	`/api/auth/me/`	Get current user profile
`PATCH`	`/api/auth/me/`	Update username or email

Register request body:

```
{
  "username": "alice",
  "email": "alice@example.com",
  "password": "StrongPass123!",
  "password_confirm": "StrongPass123!"
}
```

Login response — the `refresh` token is not present in the body; it is set automatically as an HTTP-only cookie (`refresh_token`) scoped to `/api/auth/`:

```
{
  "access": "<jwt>",
  "user": {
    "id": "uuid",
    "username": "alice",
    "email": "alice@example.com",
    "question_count": 0,
    "created_at": "2026-05-16T10:00:00Z"
  }
}
```

Documents

Method	Endpoint	Description
`POST`	`/api/documents/`	Upload a document (`multipart/form-data`, field: `file`)
`GET`	`/api/documents/`	List own documents (paginated, filterable)
`GET`	`/api/documents/{id}/`	Get document detail
`DELETE`	`/api/documents/{id}/`	Delete a document
`GET`	`/api/documents/{id}/status/`	Poll processing status
`GET`	`/api/documents/{id}/chunks/`	Debug: inspect stored vector chunks

Supported file types: `txt`, `pdf`, `xlsx`, `xls`, `csv`

List query parameters:

Param	Type	Description
`status`	string	Filter by `pending`, `processing`, `completed`, `failed`
`file_type`	string	Filter by extension
`search`	string	Search by filename
`ordering`	string	e.g. `-created_at`, `file_name`
`page`	int	Page number

Document status flow:

```
pending → processing → completed
                     ↘ failed
```

QA

Method	Endpoint	Description
`POST`	`/api/qa/ask/`	Submit a question — returns `task_id` immediately (202)
`GET`	`/api/qa/result/{task_id}/`	Poll for the answer once the task completes
`POST`	`/api/qa/stream/`	Streaming question answering (NDJSON)
`GET`	`/api/qa/activity/`	Paginated question history

Ask request body:

```
{
  "question": "What is retrieval-augmented generation?",
  "document_id": "uuid"
}
```

`question` is required. `document_id` scopes retrieval to a single document; omit it to query across all of the user's documents.

Ask response (202):

```
{
  "task_id": "d3b07384-d113-4ec7-9f2b-36d6b3a6f4c8",
  "status": "processing"
}
```

Result response — processing:

```
{
  "task_id": "d3b07384-...",
  "status": "processing"
}
```

Result response — completed:

```
{
  "task_id": "d3b07384-...",
  "status": "success",
  "answer": "RAG combines retrieval systems with generative models...",
  "sources": [
    {
      "file_name": "report.pdf",
      "page": 1,
      "excerpt": "RAG combines retrieval..."
    }
  ],
  "response_time_ms": 842
}
```

Stream endpoint returns `application/x-ndjson`:

```
{"token": "RAG "}
{"token": "combines "}
{"sources": [...], "response_time_ms": 620}
```

Activity history query parameters:

Param	Type	Description
`status`	string	`success`, `no_answer`, `error`
`document_id`	uuid	Filter by document
`page`	int	Page number
`page_size`	int	Items per page (default 20)

Testing

```
# Run the full test suite
pytest

# Run a specific domain
pytest tests/test_documents.py -v
pytest tests/test_qa.py -v
pytest tests/test_integration.py -v

# With coverage report
pytest --cov=. --cov-report=html
# Open htmlcov/index.html in a browser
```

The test suite collects 99 tests across six files and runs in under 10 seconds. All external dependencies (Ollama, PGVector) are mocked — no live services are required.

Coverage summary: 84% overall. Critical paths (views, serializers, services, models) are at 92–100%. Low coverage areas are the Ollama-dependent embedding and reranking paths (`documents/services/embedding.py`, `qa/services/reranking.py`) which are exercised via integration mocks but not unit-tested directly.

Admin Panel

The Django admin is available at `http://localhost:8000/admin/`.

Model	Features
User	View question count, manage permissions, search by username/email

Model	Features
Document	Filter by status and file type, bulk mark as pending/failed, colour-coded status badges
QuestionActivity	Read-only audit log, filter by status, sources count column, no add permission (system-generated only)